ONTOLOGY ENGINEERING | NATURAL LANGUAGE PROCESSING | TECHNICAL INTERVIEW DEFENSE

TF-IDF Formulation & Mathematical Foundations

Document Segmentation, Additive Smoothing, and Multi-Criteria Term Extraction in Domain Ontologies

1. Executive 5–6 Line Interview Defense

Deliver this structured response when asked: *"How did you calculate TF-IDF for your ontology project?"*

INTERVIEW RESPONSE SCRIPT (60-SECOND ANSWER)

"Since we were extracting domain terms from single comprehensive technical documents rather than an external multi-document corpus, standard corpus-level TF-IDF could not be applied directly. To establish a statistically sound document frequency, we decomposed each document into paragraph-level 'mini-documents.' For every multi-word candidate phrase extracted via POS patterns and n-grams, Term Frequency (TF) was its full-text count, while Document Frequency (DF) was the number of paragraph segments containing it. We then applied a smoothed logarithmic inverse document frequency formula—$\ln((1+N)/(1+DF)) + 1.0$—which penalizes pervasive generic vocabulary while amplifying localized, salient concepts. Finally, this phrase-level TF-IDF was integrated at a 40% weight alongside C-Value and linguistic boosts in a hybrid scoring pipeline to rank high-precision ontology terms."

2. Architectural Strategy: Single-Document Decomposition

The Single-Document Bottleneck

Classical TF-IDF assumes a large corpus of documents ($M \gg 1$). In single-document extraction (e.g., an aerospace technical report):

$N = 1 \implies DF(t) = 1 \implies IDF(t) = \ln(1/1) = 0$

All discriminative power collapses because every word appears in the exact same single document.

Our Solution: Paragraph Mini-Documents

In `split_into_docs(text)`, text is partitioned into paragraph segments via regex boundaries:

```
re.split(r' \s* |(?⩽[.!?])\s{2,}', text)
```

Fallback: If segments < 3, groups of 5 sentences are assembled. This creates $N \in [30, 200]$ natural semantic mini-documents.

3. Mathematical Derivation of the Smoothed IDF Formula

SMOOTHED PHRASE-LEVEL INVERSE DOCUMENT FREQUENCY

$$IDF(p) = \ln\left(rac{1 + N}{1 + DF(p)} ight) + 1.0$$

Implemented in `scoring.py` • Equivalent to Scikit-Learn `smooth_idf=True` with unit baseline offset

Component	Implementation in Code	Mathematical & Engineering Purpose
N (Universe Size)	`n_docs = max(len(docs), 1)`	Total number of paragraph segments. Protected against empty inputs with `max(..., 1)`.
$DF(p)$ (Segment Freq)	`sum(1 for d in low_docs if p in d)`	Number of segments containing the lowercase phrase. Evaluates dispersion vs. topical localization.
Ratio $ rac{N}{DF}$ (Inversion)	Penalizes pervasive vocabulary	Terms in all segments ($DF pprox N$) yield ratios near 1; localized terms receive strong multiplicative weight.
$\ln(\cdot)$ (Natural Log)	Sublinear dampening	Compresses exponential ranges, preventing 1-off outliers from eclipsing recurring key concepts.
$+1$ Smoothing (Laplace)	$ rac{1 + N}{1 + DF(p)}$	**Zero-division safeguard:** Guarantees a safe denominator if string boundaries differ ($1 + 0 = 1$).
$+1.0$ Outside Log	$\ln(\dots) + 1.0$	**Prevents term zeroing:** When $DF = N$, $\ln(1) = 0$. The $+1.0$ floor ensures $IDF \ge 1.0$ so core terms are not erased.

4. Concrete Numerical Case Study ($N = 100$ Paragraph Segments)

To demonstrate mathematical behavior, below is an exact step-by-step evaluation of four candidate term archetypes within a 100-segment technical paper:

Candidate Term	Linguistic Archetype	Frequency (TF)	Segments (DF)	Exact IDF Calculation	IDF	TF × IDF
"system"	Ubiquitous Generic Word	`120`	`100`	$\ln(101 / 101) + 1.0 = \ln(1.0) + 1.0 = 0 + 1.0$	`1.0000`	`120.0`
"flight control"	High-Level Domain Theme	`45`	`25`	$\ln(101 / 26) + 1.0 = \ln(3.884) + 1.0 = 1.357 + 1.0$	`2.3570`	`106.1`
"autonomous waypoint navigation"	Salient Technical Concept	`14`	`5`	$\ln(101 / 6) + 1.0 = \ln(16.833) + 1.0 = 2.823 + 1.0$	`3.8233`	`53.5`
"micro-Doppler radar signature"	Ultra-Specific Deep Concept	`4`	`1`	$\ln(101 / 2) + 1.0 = \ln(50.500) + 1.0 = 3.922 + 1.0$	`4.9219`	`19.7`

5. Multi-Criteria Composite Scoring Architecture

In `pipeline.py`, TF-IDF provides the **distributional relevance** pillar. It is integrated into a multi-dimensional ranking equation:

COMPOSITE CANDIDATE TERM RANKING FORMULA

$$ext{Score}(p) = 0.40 \cdot ext{C-Value}(p) + 0.40 \cdot ext{TF-IDF}(p) + 0.10 \cdot ext{Freq}(p) + 0.05 \cdot ext{AcrBoost} + 0.05 \cdot ext{PosBoost}$$

Role of C-Value (40%)

C-Value measures **internal linguistic cohesion**. It discounts phrases when they predominantly appear nested inside longer technical phrases (e.g., prevents *"flight"* from overshadowing *"autonomous flight control"* based solely on raw token frequency).

Role of TF-IDF (40%)

TF-IDF measures **external segment specificity**. Even if a phrase has high C-Value, if it is distributed homogenously across every paragraph like generic prose, TF-IDF scales it down relative to localized topical concepts.

Acronym Alignment Boost ($+1.5$)

If the initials of candidate tokens match a capitalized acronym discovered in the document text (e.g. *UAV* $\leftrightarrow$ *Unmanned Aerial Vehicle*), the phrase receives an automatic $+1.5$ boost.

Part-of-Speech Pattern Boost ($+1.0$)

Phrases matching recognized multi-word grammatical patterns (such as `(Noun|Adj)+ Noun`) verified by SpaCy/NLTK POS tagging receive an additional $+1.0$ confidence boost.

6. Methodological Comparison: TF-IDF Variants

Dimension	Standard Text Retrieval (IR)	Scikit-Learn `TfidfVectorizer`	Our Ontology Pipeline
Corpus Scope	Multi-document library ($M > 10,000$)	List of independent string documents	Single technical paper broken into paragraph segments
Extraction Unit	Single words (unigrams)	Blind n-grams ($1 \le n \le 3$)	Linguistically validated multi-word noun phrases
Normalization	BM25 document length tuning	L_2 Euclidean row vector normalization	Unnormalized magnitude, blended with C-Value
Zero Floor	$\ln(M/DF) o 0$ for corpus terms	Offset $+1.0$ ensures minimum 1.0	Offset $+1.0$ preserves foundational domain anchors

7. Production Python Implementation (from `scoring.py`)

```python
import math

def tf_idf_phrase_scores(candidates: dict, docs: list) -> tuple:
    """Computes phrase-level TF-IDF over paragraph mini-documents with additive smoothing."""
    n_docs = max(len(docs), 1)
    low_docs = [d.lower() for d in docs]

    scores = {}
    dfs = {}

    for phrase, tf in candidates.items():
        lp = phrase.lower()

        # Document Frequency: count segments containing the lowercase phrase substring
        df = sum(1 for d in low_docs if lp in d)

        # Smoothed Logarithmic Inverse Document Frequency with +1.0 floor offset
        idf = math.log((1.0 + n_docs) / (1.0 + df)) + 1.0

        scores[phrase] = tf * idf
        dfs[phrase] = df

    return scores, dfs, n_docs
```

8. Technical Interview Q&A Defense Cards

Q1: Why did you add $+1.0$ outside the logarithm rather than standard textbook $\ln(N/DF)$?
Answer: Textbook $\ln(N/DF)$ evaluates to zero whenever a term occurs in every document segment ($N=DF \implies \ln(1)=0$). In domain extraction from a specialized text, foundational anchor terms (e.g. *'aircraft'* in a flight manual) appear across almost every paragraph. Zeroing out their IDF would wipe out their TF entirely ($ ext{TF} imes 0 = 0$), eliminating the most critical domain concepts. The $+1.0$ offset guarantees that ubiquitous words have an IDF of 1.0 (retaining their full term frequency), while localized terms receive positive multiplicative boost.

Q2: Why did you segment the document into paragraphs instead of taking sentences or chapters?
Answer: Paragraphs represent coherent rhetorical sub-units: a single paragraph generally addresses a specific subtopic or mechanism (e.g., propulsion, navigation law, telemetry). Sentences are too granular (resulting in artificially high sparsity and noisy IDF), while entire sections or chapters are too coarse (obscuring localized technical concepts). Paragraph-level segmentation provides the optimal statistical granularity for document frequency.

Q3: How does this formulation prevent sub-phrase dominance (e.g., 'control' vs. 'adaptive flight control')?
Answer: TF-IDF alone is vulnerable to sub-phrase dominance because shorter words naturally have higher frequency and broader segment coverage. To solve this, we couple TF-IDF at equal weight (40%) with **C-Value**. C-Value explicitly subtracts the average frequency of longer candidate phrases in which a shorter phrase is nested, preventing constituent sub-words from outranking their more specific parent phrases.

Q4: Why not simply use Scikit-Learn's `TfidfVectorizer` for the entire pipeline?
Answer: We do use `TfidfVectorizer` for benchmark evaluation and context retrieval in our clustering stage, but for primary term extraction, `TfidfVectorizer` generates arbitrary character or word n-grams without syntactic grammar awareness. Moreover, it applies L_2 Euclidean normalization per row, which distorts term weights when aggregating scores across segments of varying lengths. Our custom function preserves raw absolute phrase salience.